

Documentación del proyecto

Se debe realizar la siguiente documentación por cada proyecto.

Fase 1. Contextualización

- ¿Qué se debe hacer?

Desarrollar un programa que simule el funcionamiento de una cola de impresión para administrar solicitudes de documentos enviadas desde varias computadoras (PC1 a PC5). La atención a los trabajos debe realizarse bajo el principio FIFO (*First In, First Out*), atendiendo primero la solicitud que llegó primero.

- ¿Qué se necesita?

Un lenguaje de programación que permita el manejo de listas/arreglos (en este caso, Python).

Una estructura de datos lineal (Cola / *Queue*).

Un límite o capacidad máxima para la cola (6 elementos).

Generación de datos aleatorios para seleccionar la computadora de origen (PC1 a PC5).

Un menú interactivo en consola para probar las operaciones de agregar, imprimir y visualizar.

- ¿Qué elementos se identifican?

Entidades: Computadoras solicitantes (PC1, PC2, PC3, PC4, PC5).

Estructura de Datos: Cola (lista) de trabajos de impresión.

Operaciones Principales: Agregar trabajo, Imprimir/Atender, Mostrar Cola.

Controles de Estado: Verificación de "Cola Llena" y "Cola Vacía".

Fase 2. Definición de los requerimientos funcionales

Se debe elaborar el listado de los requerimientos del contexto planteado

Requerimientos	
Nombre	R1-Agregar trabajo de impresión
Descripción	Selecciona de forma aleatoria una computadora entre PC1 y PC5 y la agrega al final de la cola de impresión.
Entrada	Selección de la opción 1 en el menú por parte del usuario.
Salidas	Mensaje que indica qué PC envió el trabajo o un mensaje de error si la cola está llena.
Nombre	R2-Imprimir documento
Descripción	Elimina y procesa el primer elemento de la cola respetando el orden de llegada (FIFO).
Entrada	Selección de la opción 2 en el menú por parte del usuario.
Salidas	Mensaje indicando la PC atendida (ej. "Impresión PC2") o mensaje indicando que la cola está vacía.
Nombre	R3-Mostrar cola de impresión
Descripción	Muestra en pantalla el estado actual de la cola de impresión con la lista de trabajos pendientes y la cantidad de ocupación actual.
Entrada	Actualización automática al realizar cualquier acción o selección del menú.
Salidas	Estado visible de los elementos en la lista y la capacidad de la cola.

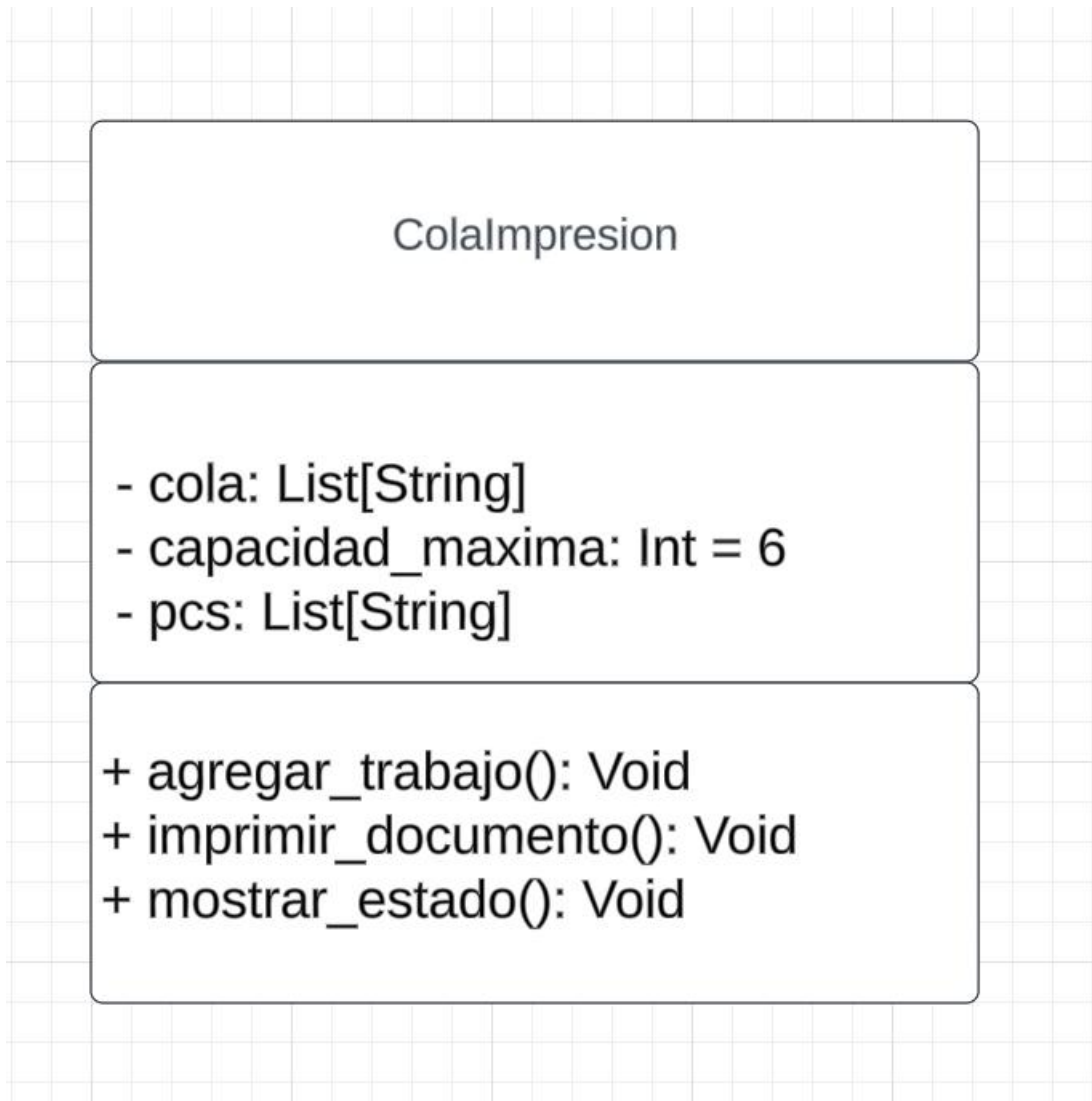
Nombre	R4-Verificar cola llena
Descripción	Compara el número de elementos en la cola con el límite máximo (6 elementos). Si la cola alcanza el límite, bloquea nuevos registros.
Entrada	Intento de agregar un nuevo trabajo (Opción 1).
Salidas	Mensaje: "La cola de impresión está LLENA".
Nombre	R5-Verificar cola vacía
Descripción	Verifica si la lista tiene 0 elementos antes de intentar eliminar un elemento.
Entrada	Intento de imprimir un documento (Opción 2).
Salidas	Mensaje: "La cola de impresión está VACÍA".

Para la documentación de cada requerimiento funcional utilizar el siguiente esquema descriptivo.

Nombre	R1 – Nombre del requerimiento
Descripción	Especificar de forma detallada lo que hace el requerimiento
Entrada	La información que espera recibir
Salidas	El resultado esperado

Fase 3. Diseño

Determinar las clases que conforman el sistema, junto con los atributos y los métodos y crear el diagrama de clases UML, así como sus relaciones de herencia (si aplica).



Fase 4. Codificación

Se deben escribir los programas fuentes necesarios, basados en el diagrama de clase de la fase anterior:

```
1
2 import random
3
4 cola = []
5 capacidad_maxima = 6
6 pcs = ["PC1", "PC2", "PC3", "PC4", "PC5"]
7
8 opcion = ""
9
10
11 while opcion != "3":
12
13     print("\n-----")
14     print("Estado actual de la cola:", cola)
15     print("Espacio ocupado:", len(cola), "de", capacidad_maxima)
16     print("-----")
17
18
19     print("1. Agregar trabajo")
20     print("2. Imprimir documento")
21     print("3. Salir")
22
23     opcion = input("Selecciona una opción: ")
24
25
26 if opcion == "1":
27     if len(cola) >= capacidad_maxima:
28         print("\n La cola de impresión está LLENA")
29     else:
30         pc_aleatoria = random.choice(pcs)
31         cola.append(pc_aleatoria)
32         print("\nSe agregó:", pc_aleatoria)
33
34
35 elif opcion == "2":
36     if len(cola) == 0:
37         print("\n La cola de impresión está VACÍA")
38     else:
39         atendido = cola.pop(0)
40         print("\n🖨 Impresión", atendido)
41
42
43 elif opcion == "3":
44     print("\n¡Hasta luego!")
45
46 else:
47     print("\nOpción no válida, intenta de nuevo.")
```

Fase 5. Pruebas.

Aquí se debe:

```
-----  
Estado actual de la cola: []  
Espacio ocupado: 0 de 6  
-----  
1. Agregar trabajo  
2. Imprimir documento  
3. Salir  
Selecciona una opción: 2  
  
La cola de impresión está VACÍA  
  
-----  
Estado actual de la cola: []  
Espacio ocupado: 0 de 6  
-----  
1. Agregar trabajo  
2. Imprimir documento  
3. Salir  
Selecciona una opción: []
```

Se agregó: PC4

Estado actual de la cola: ['PC3', 'PC4']
Espacio ocupado: 2 de 6

1. Agregar trabajo
2. Imprimir documento
3. Salir
Selecciona una opción: 1

Se agregó: PC5

Estado actual de la cola: ['PC3', 'PC4', 'PC5']
Espacio ocupado: 3 de 6

1. Agregar trabajo
2. Imprimir documento
3. Salir
Selecciona una opción: 1

Se agregó: PC2

Estado actual de la cola: ['PC3', 'PC4', 'PC5', 'PC2']
Espacio ocupado: 4 de 6

1. Agregar trabajo
2. Imprimir documento
3. Salir
Selecciona una opción: 1

Se agregó: PC4

Estado actual de la cola: ['PC3', 'PC4', 'PC5', 'PC2', 'PC4']
Espacio ocupado: 5 de 6

1. Agregar trabajo
2. Imprimir documento
3. Salir
Selecciona una opción: 1

Se agregó: PC1

Estado actual de la cola: ['PC3', 'PC4', 'PC5', 'PC2', 'PC4', 'PC1']
Espacio ocupado: 6 de 6

1. Agregar trabajo
2. Imprimir documento
3. Salir
Selecciona una opción: